Trace observability: the interception bus, diagnosis, and coverage

TBK SPEC 25:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 25:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword 4

1. Laws 5

2. The interception bus (`tfs::trace`) 6

3. The event envelope (schema v1) 7

4. `tebako trace run` — discovery 8

5. `tebako trace explain` — diagnosis 8

6. `tebako trace cover` — coverage 9

6.1 The three-layer model (normative) 9

6.2 The outside format — retrace-shaped JSON (SSOT: retrace) 9

6.3 The correlator — self-contained Rust 10

6.4 The contract with the retrace project 11

7. Testing and gates 12

8. Phasing 13

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

Foreword

Status: PARTIAL (phase T1 shipped 2026-08-19: the `tfs::trace` interception bus + `docs/spec/schemas/trace-event.yaml` + `tebako trace run`; phase T2 shipped 2026-08-20: the `spawn/resolve` emission — the preload's posix_spawn surface and the driver's image-triple resolution; phase T3 shipped: `tebako trace cover` (§6.3) 2026-08-20 — the escapes report, golden-parity with retrace's shared correlate fixtures — and 2026-08-21 the procmon converter (`tebako trace import procmon`, §6.2), byte-parity with upstream procmon2retrace on the golden 06 fixture; phase T4 shipped 2026-08-21: `tebako trace explain` (§5) — the capture replay with the signature table in data. All four phases' CODE is on main; the PARTIAL remainder is dogfood, not code: the §8 legs (retrace libc-layer on both platforms; the kernel-layer leg as §6.4 lands) and the real incident-13 captures' archival into the explain fixtures. Owner-signed 2026-08-19 — architecture A: the in-tfs event bus + `tebako trace` front-ends; outside capture rides the three-layer producer model of §6.1, with retrace as the libc-boundary layer on BOTH platforms).

A payload author virtualizing an application on tebako must answer three questions, and today each is answered by hand-rolled one-off diagnostics (the incident-13 msys dogfood chain: four CI rounds of `eprintln` gates and in-probe bisection to name one Windows loader failure):

1. Discovery — what does my application touch, so what must I declare? (mounts, `needs:`, `materialize:`, entrypoints)
2. Diagnosis — my packaged application fails; which hop failed? (mount → manifest → resolve → materialize → OS bind)
3. Coverage — is anything my application does NOT virtualized? (raw syscalls, unpatched IATs, loader-internal probes — the escapes)

This spec defines one substrate and three front-ends:

- The interception bus (§2): every tebako interception point emits one structured event. The bus lives in the Rust `tfs` crate — the shipping VFS implementation on every platform — so it is cross-platform and runtime-independent by construction: events name syscall/loader-level facts (paths, verdicts, errnos), never language facts. Ruby,

© 2026 The Tebako Project

Julia, and Python payloads are equally covered because every runtime bottoms out at the same libc/loader boundary the bus instruments.

- `tebako trace run` (§4): discovery — collect a run's events and emit a suggested manifest.
- `tebako trace explain` (§5): diagnosis — replay a capture into the hop chain and name the first red hop.
- `tebako trace cover` (§6): coverage — correlate the inside stream with an outside capture and report what escaped.

1. Laws

1. **Observability never gates.** A trace-channel failure degrades to a loud stderr note; the payload run proceeds and its exit code is unaffected. A tracer must never become a failure mode.
2. **Runtime-independent.** Events name OS-level facts only. No event may carry a language-runtime concept (a gem, a module, a Julia package) — runtimes are payloads; the bus sits below them.
3. **Cross-platform inside; layered outside.** The inside stream is identical on linux, macOS, and windows. The outside view is the three-layer producer model (§6.1): the libc boundary is retrace's on BOTH platforms (preload on POSIX; the preload-msvc / preload-mingw inline-hook backends on windows, retrace ADR-0009); kernel-level producers (ptrace, eBPF, ETW/procmon) join ONLY as retrace-shaped JSON, via bridges or offline converters (§6.2). Every outside source is an import format, never a runtime dependency.
4. **No shell-outs.** Captures are files parsed in-process. Arming or reading a trace never spawns a tool (spec 00 invariant 1).
5. **The correlation law.** The inside stream can never prove its own completeness: an escape is invisible from inside by definition (macOS 15's dyld rpath probes never reach the interposed open — the exec-closure walk exists because of this). "What isn't tebako intercepting" is answerable ONLY by subtracting the inside stream from an outside capture — and only up to the outside capture's own layer (§6.1).
6. **MECE with spec 23 §8.** The `record` policy mode already owns host-path needs discovery (the audit journal → `tfs needs --from-journal`). This spec does not re-invent it: the jail channel of the bus IS that journal, formalized; §4's discovery extends the draft to the axes §8 does not cover (`materialize:`, entrypoints, runtime deps).
7. **Human gate.** A generated suggestion never edits a manifest by itself (spec 23 §8's law, unchanged).

© 2026 The Tebako Project

2. The interception bus (`tfs::trace`)

Emission points — exactly the interception points the driver/TFS already implements, one event per decision:

TABLE 1

op	emitted where	verdict values
`mount`	mount table insert/remove	ok / `error:<errno>`
open / `stat`	path dispatch (spec 11)	`image:<mount>` / `host` / `denied:<rule>` / `error:<errno>`
`dlopen`	`dlmap2file` / `dlalias2file`	`materialized:<host-path>` / `host` / `error:<errno>`
`exec` / `spawn`	exec routing (spec 17/22) — one routing decision, two syscall surfaces: `exec` is the execve surface (process replacement), `spawn` the posix_spawn surface (a child is created — the correlator's process-tree signal)	`routed:<entry>` / `host` / `error:<errno>`
`materialize`	exec-cache extraction	`ok:<host-path>` / `cache-hit` / `error:<errno>`
`jail`	policy bind / check (spec 08, 23 §8)	`allow:<rule>` / `deny:<rule>` / `record`
`resolve`	the driver's `--tebako-image <path>:<slot>:<mount>` resolution (trailer probe → whole file or slot region — the payload/slot identity the correlator matches outside reads against). PLANNED: the L3 runtime/payload cache resolution (tebako-resolve — loader-side, outside the runtime process; its channel story lands with the phase that needs it)	`whole` / `slot:<n>` / `error:<errno>` — PLANNED L3 half: `cache` / `fetched` / `error`

The `dlopen` event's `detail` carries the closure walk as structure — the parsed format, the dep list, each dep's resolve verdict (`materialized` / `host-system`) — what incident 13's round 4 added as `eprintln` lines. Those gates convert to events; `TEBAKO_DEBUG_TFS` remains as the human-readable degraded view over the same emission points.

Arming. `TEBAKO_TRACE=<host-path>` (env) or `--tebako-trace <host-path>` on the driver argument grammar (spec 17 — the env-scrubbed handoff rides the argument, the lesson of incident 13 round 5). The flag is an additive optional driver argument: spec 17's grammar extends by option, never by version bump (the trailer-flags rule, spec 02). Both set: the argument wins. The channel is opened and pre-registered at driver boot, before any mount — writes are append-only, process-internal, and NEVER policy-gated (the journal channel's discipline, `crates/tfs/src/journal.rs`, already proven under deny).

© 2026 The Tebako Project

Cost when off. Disarmed, an emission point is one `Option::is_none()` branch. The bus compiles into the runtime/driver side only; the bootstrap's < 3 MB size gate is untouched (spec 00 invariant 2).

Children. Spawned children re-derive the channel at their own driver boot on BOTH platforms: `TEBAKO_TRACE` is in the inherited environment, the child's driver arms, and its events append to the same file. (The fd-inheritance spelling this section used to carry is not the mechanism — env re-derivation covers both platforms uniformly. On fork, the already-open fd IS inherited until the child's own arm, so pre-arm child events still land in the same channel.) All writers append atomically; events may interleave — every event carries `pid`/`tid` so the front-ends regroup.

3. The event envelope (schema v1)

One JSON object per line (JSONL; streamable, diffable, robust to a crashed tail). Authored configs stay YAML (invariant 6); the event STREAM is telemetry and is JSON. The versioned schema lives at `docs/spec/schemas/trace-event.yaml` (lands with phase T1); the normative shape:

FIGURE 1

```
v: 1                    # envelope version — additive only,
never bumped
ts: <rfc3339µs>         # driver clock
pid: <int> ; tid: <int>
op: <table §2>
path: <vfs-or-host path, exactly as dispatched>
verdict: <table §2>
detail: { ... }         # per-op: dlopen deps, jail rule,
exec argv0, …
dur_us: <int>           # the interception's own cost
errno: <int, optional>  # when verdict names an error
```

CI validates every emitted event against the schema (a property test drives the whole op matrix and validates the stream).

© 2026 The Tebako Project

4. `tebako trace run` — discovery

Runs an application composition under `policy: record` with the bus armed, then synthesizes a **suggested manifest** — a commented YAML fragment, never applied (law 7):

- host reads → `needs: /jail: ro` grants (this half is spec 23 §8's generator, unchanged — the bus's jail channel feeds it);
- host writes → rw grants (strongest-observed-op wins, §8's rule);
- a VFS-resident file read through a RAW host fd (a native library's own `fopen` — the exec-cache answer consumed it) → a `materialize:` candidate — the sassc class-R case, caught automatically instead of discovered by a failing import;
- an in-image `dlopen` whose deps all resolve in-image → a NOTE "closure-covered — nothing to declare";
- a spawned host-absolute executable → an entrypoint/runtime-dep note;
- floor, store, and exec-cache paths are excluded (§8's exclusion law).

Output: the draft fragment on stdout (or `--out`), each entry carrying its `why:` evidence line (`observed: <n> events, first at <ts>`). The author reviews, flips ro↔rw, deletes noise, fills `why`.

5. `tebako trace explain` — diagnosis

Replays a capture (or a live run's stream) into the hop chain — mount → manifest read → resolve → materialize → OS bind — and prints the first hop whose verdict is red, with the signature table seeded from the incident corpus:

TABLE 2

signature in the stream	named hop
no `mount` event for the env image + prelude-class child stderr	env image never mounted (handoff env lost)
`dlopen` verdict naming the OS loader's module-not-found (windows error 126; the POSIX `dlopen` ENOENT analogue) with every dep `materialized|host-system`	the OS bind itself — bisect candidates printed
`jail` deny immediately preceding a dependent open error	policy denial (the EACCES class)
`materialize error`	exec-cache write failure

© 2026 The Tebako Project

The table lives in data (a YAML map in the CLI crate), extended as incidents teach new signatures — never hard-coded into the bus.

6. `tebako trace cover` — coverage

The escapes report (law 5). Inputs: an inside capture (the bus) and an outside capture of the same run.

6.1 The three-layer model (normative)

TABLE 3

layer	POSIX	windows	who sees it
inside the VFS	the tfs bus (§2)	the tfs bus (§2)	tebako
libc boundary	retrace preload	retrace inline hooks (preload-msvc / preload-mingw, retrace ADR-0009)	retrace — both platforms
kernel syscall	retrace ptrace backend; strace → strace2retrace (converter, §6.2); eBPF bridge	ETW / procmon (via converter, §6.2)	kernel tracers

Escapes split into two classes at the libc boundary:

- libc-routed escapes — the application called `open(2)` / `CreateFileW` directly, below tebako's interposition but through libc. Visible to retrace on BOTH platforms today.
- sub-libc escapes — raw `syscall(2)` / `NtCreateFile`, loader-internal probes (the macOS dyld rpath class). Invisible to libc hooks by construction; only a kernel-layer producer sees them.

A coverage report is only as strong as its outside capture's layer: a libc-boundary capture certifies libc-routed escapes only; sub-libc certification requires a kernel-layer capture. The report MUST name the producing layer of its outside input.

6.2 The outside format — retrace-shaped JSON (SSOT: retrace)

The outside capture format is the retrace trace document, owned and versioned by the retrace project (riboseinc/retrace). Tebako never authors a competing outside format. Every producer normalizes INTO that shape; the correlator stays format-uniform:

© 2026 The Tebako Project

- retrace preload / inline hooks — native JSON output today (one JSON array document per capture).
- retrace ptrace backend — native, POSIX kernel layer.
- eBPF bridge (`ebpf-bridge/`) — emits retrace-shaped JSON.
- procmon / ETW (windows kernel layer) — an OFFLINE converter (procmon CSV/XML export → retrace JSON), the same decision as the Frida/eBPF bridges: producers normalize into the format, never the reverse. The converter ships in the tebako CLI as an import verb (`tebako trace import procmon <csv>`), pure Rust, in-process (law 4).

6.3 The correlator — self-contained Rust

`tebako trace cover` implements the correlation IN the tebako CLI, in safe Rust: canonicalize host paths on both sides (same atoms as spec 23 §8's generator); subtract every outside-observed op that an inside event already accounts for; the remainder is the escapes list, grouped by surface class (fs / exec / dlopen / spawn), with a per-class coverage percentage and the producing layer named (§6.1). Exit code 0 = no escapes, 1 = escapes found — CI-gateable. Intentionally-unserved surfaces (documented passthroughs, e.g. the platform floor) are declared in the composition so the report separates KNOWN-UNSERVED from ESCAPE.

The shipped surface (phase T3, the correlator half).

FIGURE 2

```
tebako trace cover --inside <tfs.json|jsonl> --outside
<retrace.json>
                   --prefix <path> [--pid N] [--window SECS]
                   [--exclude-probes] [--json] [--layer libc|
kernel]
```

- stdout is the golden contract: one `escape <path> func=<f> tid=<t> pid=<p> class=<probe|read|write|none>` line per escape (retrace's report grammar since its v2.9.0), or retrace's escape array under `--json`. Byte-compared in CI against retrace's shared golden tree (`tools/correlate/golden/`, copied byte-verbatim to `crates/tebako-cli/tests/fixtures/correlate/`) — every case, stdout bytes and exit code. Nothing else ever writes stdout for this subcommand (the version banner rides stderr).
- stderr (outside the golden contract) carries the summary (`inside=N entries, M paths; prefix=P; escapes=E`), the escapes grouped by surface class with per-class coverage percentages, and the producing layer named by `--layer` (default `libc` — §6.1's honesty rule: the report certifies its named layer only; sub-libc escapes are UNCERTIFIABLE under a libc-boundary capture).

© 2026 The Tebako Project

- **Exit codes** are retrace-correlate's: 0 no escapes, 1 escapes found, 2 usage or I/O error.
- **The kernel is retrace's** (the `retrace-correlate` port): the tolerant stream scan (array document / JSONL / truncated tail), any-depth path extraction, NT-form normalization (`\??\`, `//?/`, `\\?\`, the `\Device\HarddiskVolumeN` drive guess), drive-letter-insensitive path comparison with a case-SENSITIVE component-boundary prefix test, pid/time-window coverage (a pid-less entry is a wildcard; the bus's string `ts` never feeds `--window` — window mode takes retrace-shaped inside captures with numeric `time`), the name-keyed probe/write classification (a NAME-NOT-FOUND probe on an unserved under-prefix path IS an escape — existence is information), one escape per outside entry (the first under-prefix uncovered path in field order), `--exclude-probes` for the jail-grant policy. Documented deviations, error paths only: a non-numeric `--pid`/`--window` and an unknown `--layer` are named usage errors (exit 2) where upstream's atol/atof silently read 0 (spec 00 law 9).

The correlator is tebako-owned because the invariants leave no other home: invariant 1 (no shell-outs in shipped artifacts) forbids invoking a retrace binary; invariant 3 (C/C++ only in dwarfs-t) forbids linking one. The correlation kernel (normalize + set difference + threshold) is small and pure. If the retrace project ships its own `retrace-correlate` for the generic market, the two implementations assert parity on shared golden fixtures in CI — invariant 10's parity escape hatch: the FORMAT is retrace's single source; the ALGORITHM has two implementations that must agree. (retrace's `retrace-correlate` landed ahead of this phase — the golden tree above is that parity hatch, exercised.)

6.4 The contract with the retrace project

Verified against riboseinc/retrace @ v2.6.1 (d06c6af); later shipments tracked below the table (the full correspondence lives in the work queue). Tebako phases coverage features as these land upstream; none block T1/T2:

TABLE 4

prerequisite	state today	tebako impact until it lands
`retrace-audit` + `path_contains` policy op	SHIPPED (tools/audit-converter; property-tested — P-POLICY-CONTAINS-SOUNDNESS)	the policy-matching half of coverage exists; usable on outside captures immediately
pid + tid per log entry	SHIPPED (v2.7.0) — the correlator joins on pid/tid when present	multi-process compositions regroup the outside stream
ntdll-depth hooks (`NtCreateFile` / `NtOpenFile` / `LdrLoadDll`) + NT-path normalization (`\??\C:\…`)	ABSENT at v2.6.1 — the windows backends interpose at the ucrt layer; a native payload calling Win32 directly already escapes that layer TODAY (libsass's importer reads via raw `CreateFileW` with `\\?\` prefixes — invisible at ucrt)	windows sub-libc escapes stay invisible until both the hooks and a converter exist

© 2026 The Tebako Project

prerequisite	state today	tebako impact until it lands
streaming / JSONL output	today one JSON array document per capture (leading-comma emission; a crashed tail invalidates the document)	the correlator tolerates a trailing partial record
`retrace-correlate` (the set-difference tool)	SHIPPED (v2.7.0, with the shared golden tree incl. 06-libsass-importer; the report grammar stabilized in v2.9.0)	tebako ships its own (§6.3) and asserts byte-parity on the shared tree in CI — the parity hatch is exercised
ptrace-backend param deref (arch_spec)	ABSENT at v2.14.0 — the ptrace backend ships no arch_spec of its own: aarch64 entries carry the syscall NAME with NIL params (`openat(AT_FDCWD, "/tmp/x", O_RDONLY)` arrives as `"path":"(nil)"`), and on x86_64 the wild frame deref SIGSEGVs on attach (tebako CI run 32451535675, job 96680888031)	the linux kernel-layer dogfood rides strace + upstream's OWN `strace2retrace` converter (the §6.2 normalizer pattern — same kernel truth, real paths) until this lands or the eBPF bridge ships
windows live capture (win-run injection + ucrt inline hooks)	NEVER smoke-tested upstream at v2.14.0 (msys.yml exercises only the library scaffold + headers); BROKEN on windows-latest in tebako CI (run 32453723170, job 96686997021): the hook installation completes in the child (`hooked 'fopen' ...` diagnostics print), then `LoadLibraryA(retrace.dll)` fails inside `retrace_core_boot()` — the loader converts a DllMain crash into `ERROR_DLL_INIT_FAILED`, the inject thread's exit code reads 0, and win-run reports "failed to launch" (`win_common/inject.c:73`)	the windows libc-layer dogfood is a prerequisite probe: a standalone control isolates our half, a `RETRACE_LOGGER_RING=0` candidate pin gets the full chain gates when it captures (the Min GW build compiles the lock-free ring the MSVC build stubs out), and otherwise the leg pins the documented broken signature — drift in either direction fails it. The procmon converter (§6.2) carries the windows chain certification meanwhile

7. Testing and gates

- Golden fixtures: the incident-13 dogfood captures (real `[tfs]` streams) converted to the schema must round-trip through the validator and through `explain` (whose red hop must match the incident's hand-derived answer).
- Property: driving the public tfs op matrix emits exactly one schema-valid event per op.
- Correlator: synthetic inside/outside pairs with deliberate escapes — a libc-routed fixture (a direct `fopen` below the interposition) under a retrace preload capture, and a raw-`syscall(2)` fixture binary (built by CI, not shipped) under a KERNEL-layer capture — must be caught at 100%. The raw-syscall fixture paired with only a libc-boundary

© 2026 The Tebako Project

capture must report the escape as UNCERTIFIABLE, not as covered (§6.1's honesty rule).

- Converter: a procmon CSV fixture converts to retrace-shaped JSON the correlator consumes identically to a native retrace capture.
- Parity (when `retrace-correlate` lands upstream): the shared golden fixtures produce identical escapes reports from both correlators.
- The factory's msys dogfood converts from TEBAKO_DEBUG_TFS gates to the bus as phase T1's dogfood — the harness stops hand-rolling diagnostics.
- Size/hygiene: bootstrap gate untouched; no new dependencies outside the workspace (`tebako-json` for emission); `unsafe` nowhere (the bus is safe Rust over the journal's fd).

8. Phasing

TABLE 5

phase	contents	dogfood
T1	the bus + schema + `run` (discovery)	the msys dogfood rides the bus; a payload author onboards a fresh gem without hand diagnostics
T2	the `spawn`/`resolve` emission: the preload's posix_spawn surface reports `spawn` (a child-process decision — the correlator's process-tree signal), and the driver emits one `resolve` event per `--tebako-image` triple (the payload/slot identity: whole / `slot`:<n> + offset/size/mount, or the named failure class)	a traced boot of a broken composition names the failing triple in the capture itself — no stderr spelunking; a traced run's spawns regroup by pid
T3	`cover` (certification) + the procmon converter	`cover` SHIPPED 2026-08-20 (the §6.3 correlator + the shared golden tree green in CI); the converter (`import`, §6.2) SHIPPED 2026-08-21 (byte-parity with procmon2retrace on the golden 06 case, the converted stream driving cover to expected.txt/exit.txt). Remaining dogfood: a leg under retrace on both platforms (libc layer); a kernel-layer leg (ptrace/eBPF on linux, procmon-converter on windows) as the §6.4 prerequisites land; escapes report in CI
T4	`explain` (diagnosis)	SHIPPED 2026-08-21: the replay + the signature table in data (`explain-signatures.yaml`), fixtures replaying the incident classes to their hand-derived

© 2026 The Tebako Project

phase	contents	dogfood
		answers (§7's gate shape). Remaining dogfood: replay the real incident-13 captures; the named hop matches history

(Re-phased 2026-08-20: emission completeness precedes the front-ends — the original T2 (`explain`) is T4, the original T3 (`cover` + the converter) stays T3, and T2 is the spawn/resolve emission the correlator's inputs depend on.)

Each phase ships behind its own PR chain against this spec; the spec moves PLANNED → PARTIAL → SHIPPED per phase (spec 00's status law).

© 2026 The Tebako Project